資料治理年會-技術場講座重點筆記

第一場：你的資料庫裡有幾個王大明？從補習班學生資料談 **Entity Resolution**

講者：陳家宏｜睿博資訊（**REPLWARE**）執行長

一句話重點：資料庫裡有好幾個同名同姓的人，分不清是同一人還是不同人，這個問題叫 Entity Resolution。

問題情境

- 補習班約 2 萬名學生，轉分校時重複註冊，同步到總公司後出現兩筆資料。
- 同樣的問題也出現在廣告受眾資料（Google、Facebook 等平台對不起來）、企業併購後的客戶合併、行銷重複發送或漏發。
- 本質不是資料量，而是資料品質。它也不是單純去重，因為去重的前提是你已經確定是同一筆。
- 實體錯了，AI 的輸出就錯，客戶實體是資料治理中最關鍵的一塊。

為什麼規則式不夠

- 地址很髒（戶籍地和居住地不同），結果只有信或不信，沒有 0.5 或 0.7 的信心程度。
- 規則寫太嚴會漏配，寫太鬆會誤配。

解法：**Fellegi-Sunter** 演算法（**1969**）

- 先用姓名加生日縮小範圍，2 萬筆縮成約 100 到 300 筆。
- 每個欄位估兩個機率：M（同一人時相符的機率）、U（不同人時相符的機率）。
- 起始值不必擔心，演算法會迭代到收斂。
- 結果分三種：高機率視為同人、低機率視為不同人、中間地帶交人工處理。

工具：**Splink**（**Python** 套件，源自英國司法部）

1. Blocking：只比較姓名生日相同的那幾百筆，避免 2 萬乘 2 萬。
2. 估計 M 與 U。
3. 計算配對機率，例如 A 和 B 是 0.92，B 和 C 是 0.88。
4. Clustering：用圖論的連通概念，A≈B 且 B≈C，就歸為同一群。

落地要想的三件事

1. Blocking 用哪些欄位。
2. M 和 U 的起始機率怎麼設，這依賴領域知識，例如地址相同不代表同一家人，有人借戶籍。
3. 中間地帶（例如機率約 0.6）如何人工處理。

這是傳統機器學習，不是大型語言模型，一台筆電就跑得動。

什麼時候不必用 **Splink**

- 已有唯一識別碼：直接 join。
- 量小（約 300 筆）、不常做、影響小：人工或寫 SQL 處理。
- 需要用的情況：缺唯一識別碼、量大、做錯會被罰、每週都要做。

自我診斷

- 有沒有人問過「系統裡怎麼有兩個一樣的人」。
- 行銷是否對同一人重複發送或漏發。
- 跟人數有關的報表是否對不上。
- 是否缺少 UID。

帶回家的一句話：

細節會忘沒關係，記得「資料庫有好幾個王大明，這叫 Entity Resolution」。

第二場：**From Data Platforms to Decision Capability**

講者：林基玄｜捷通軟體（**JetFusion**）顧問總監

一句話重點：很多組織升級了資料架構，卻沒升級決策架構；AI 時代的目標不是 Data Ready，而是 Decision Trust（決策信任）。

核心論點

- 這些年投資了資料倉儲、資料中臺、CRM、AI，平臺上線了，但業務仍依賴 IT，管理層也沒看到決策變好。
- 平臺能集中資料、提升擴充性、支援 AI，但不會自動回答決策信任的問題。

三個親身案例

案例	發現	對應的層次
銀行 DW／CRM 轉型	約 113 萬客戶被標為不可接觸，最大宗來自一條多年前的遺留規則，沒人擁有、沒人檢討。重整後約 90 萬人可重新經營	決策所有權
重大事件調查	靠約 25 年前設計好的加密、去識別化、自動控制與 audit trail，能快速還原資料來源與使用紀錄	決策可追溯
事件式即時行銷	刷卡後 5 秒內完成判斷與簡訊觸達，回應率兩位數，形成事件、判斷、行動、回饋的閉環	決策執行

- 信任不是宣告出來的，是設計出來的；出事後才補檔案、補流程叫補救。
- 決策基礎建設不是新概念，新的是參與決策的不只有人，還有 AI。

AI 改變了什麼

- 以前人會補位，多問一圈，用人的摩擦力吸收資料與規則的模糊。
- AI 不會懷疑規則，只會忠實、快速、大規模地執行，因此會放大既有體質：成熟能力被放大，治理缺口也被規模化。

決策信任的六個檢查問題

1. 關鍵數字有沒有主人？ 主人是能定義、解釋、裁決、更新的人或角色，不是信箱或部門名稱。
2. 決策路徑能還原嗎？ 要從資料、規則、模型、人的判斷一路追到最終行動，不只是 table 到 table 的 lineage。
3. 治理政策有 **AI** 這一層嗎？ 要管 AI 能否基於某目的使用資料、只能建議還是能直接觸發、何時必須人工介入。規模放大卻沒有治理，那不叫成長，叫騷擾。
4. 判斷能被辯護嗎？ 追溯不等於辯護，要能向主管機關與客戶說明為何用這個資料、規則、模型，只列 feature importance 不夠。
5. 規則與模型幾個月後還有效嗎？ 需要效能監控、drift detection、結果回饋，以及 challenge、override、stop 機制。
6. 人與 **AI** 判斷不一致時，誰裁決、誰負責？ AI 可以參與決策，責任不能沒有主人。

對技術團隊的意涵

- Metadata、lineage、識別碼與版本紀錄，決定決策能否被重建、解釋與辯護。
- 技術人的責任從「把資料送到對的地方」變成「把決策信任的條件設計進系統」。

結語：以前問「會議室那個數字可以相信嗎」，答錯代價有限；現在可能是以機器速度執行錯誤。資料治理不是為了管資料，而是為了建立決策信任。

第三場：從效能到高效益：資料庫雲端遷移後的 **FinOps** 治理與自治演進

講者：許庭魁（**Jerry**）｜華碩電腦 開放雲端 **infra lead**

一句話重點：雲端最大的浪費是過度配置，只有持續調校能讓你不再浪費；折扣只是讓你便宜地浪費。

背景

- 地端的 CAPEX 到了雲端變成每月 OPEX，所以效能調校直接等於省錢。
- 講者分析近十幾個跨 AWS、Azure、GCP 的案子，排除 LLM 後，資料庫占雲端 TCO 的比重很大。有報告指出約 32% 的雲端支出是過度配置。
- 資料庫貴的原因：商業授權貴，且因交易一致性的特性，是最後才能擴充的一層。

FinOps 的定義

- 一種文化實踐，讓工程、財務、業務共同關注雲端的商業價值；架構的每個決策都決定資產的持有成本。

財務治理的順序：由內而外

1. 先效能調校。

2. 再選對規格，避免過度配置。
3. 最後用預留（Reserved）等折扣鎖定。

核心心法是 **Performance by Design**：系統 80% 以上的效能瓶頸來自不良的資料庫設計。

DBA 為什麼會過度配置

1. 寧可多開，不可出事，照搬地端規格。
2. 為尖峰（雙十一、Black Friday、攻擊）買保險。
3. 資源耦合：大表需要大量快取，只好堆 CPU。
4. 擴容耗時：案例中縮放一次要 6 小時。

案例

- 上雲前壓測發現 cursor 造成瓶頸，改寫後查詢從 0.5 秒降到約 0.054 秒，約快 10 倍。
- 單一 instance 月費約 5,900 美元，調校後省約 50%，再搭配三年預留，進一步降低。服務有十幾個 instance，省下的金額要再乘上去。

雲端資料庫最佳化三步驟

1. 先清理：檢查每個查詢與執行計畫是否合乎 best practice。
2. 找到適合雲端的 baseline：地端會把 CPU 壓榨到很低的閒置，雲端的 CPU 基準線約設在 30 到 40%。
3. 以調校結果決定規格，再鎖定長期折扣。

自治演進：**AI Agent** 擔任 **DBA** 助手

- 用 AI 讓一個下午就能分析完所有 SQL，連重構後的程式碼都準備好。
- 採用設計思考的雙鑽石模型（找對方向、把事情做對）。
- 安全架構：Agent 不直接碰資料庫，中間用 **MCP** 隔離，只能使用預先定義的工具，並經人同意才能操作。
- 用途：主動巡檢（例如 300 台資料庫，上班前巡完並匯報）、檢查連線與備份、自動分析慢查詢並產出改善報告，DBA 審查後再派工。
- 挑戰：最小權限、人的審查、系統隔離，以及最重要的 **rollback plan**，必須先準備好。

資料架構師的 **FinOps** 思維

- DBA 從被動維運，轉為 FinOps 的貢獻者，讓工程、業務、財務對同一項資產說同一種語言。
- 關鍵詞：Value by tuning。DBA 的關注點從「不能掛、不能慢」變成「效能與成本」，成果會直接反映在下個月的帳單。

Q&A 重點：有人問能否把 OLAP 查詢從 OLTP 資料庫拆出去。講者回答可透過 replication 與讀寫分離，把 OLAP 查詢導到唯讀副本，他認為是很好的做法。他也提到一般企業很難配置足夠的 DBA。

第四場：當「客戶」有十種定義：用 **Metadata** 打造 **AI-Understandable** 的資料

講者：陳威帆（**William**）｜核桃運算 產品暨 **AI** 與數據服務總監

一句話重點：AI Agent 就像公司新人，資料沒描述清楚時它只能猜；Metadata 不再是給人看的說明檔，而是 AI 的理解基礎。

問題

- 光是「客戶」就有多種定義（新客、活躍客、付費客、開票對象），Agent 不知道你問哪一種。
- 換更強的模型有差，但不是關鍵。2026 年報告顯示 95% 的企業說不出 Copilot 類服務的財務回報，44% 暫停訂閱；成功的企業花 60 到 80% 的力氣整理資料。

Agent 要能做事的四個要點：找得到、讀得懂、算得穩、可追溯。

三個 **Agent** 會算錯的情境

1. 沒定義，**Agent** 自己挑：客單價至少有三種算法（除以訂單數、下單客戶數、總客戶數），問 1,000 次能每次一致嗎？
2. 公式對、答案錯：毛利率應該是毛利總和除以營收總和（例子中約 5.45%），Agent 卻可能把兩個毛利率直接平均。
3. 語法對、結果不合理：庫存量是每日快照，直接加總會膨脹到約 365 倍。

這不只是幻覺，而是 Agent 用對的公式，卻不知道什麼能這樣算，還在你沒注意時替你做了決定。

微軟實驗

- 給 Agent 一份約 4KB 的 metadata（粒度、業務定義、彙總方式、關聯、時間範圍），準確率從 48% 提高約 20 個百分點，三種模型都有提升。
- 不是給越多越好，標準是「可執行」，像寫程式一樣讓 Agent 不用再猜。例如欄位值 1、2、3 必須寫明各自代表什麼。

讓 **AI** 算得跟 **BI** 一樣可靠：**Semantic Layer**

- 固定指標 BI 仍然最快，動態問題才需要 AI，兩者不必互相取代。
- Semantic layer 包含找得到、讀得懂、算得穩，它是治理概念，不只是技術層。
- 做法是把公式與彙總規則寫給 Agent，例如客單價三種情境各對應哪個公式、客戶數在哪些層級 group by 合理、哪些會重複計算。這樣全公司對同一個問題就有唯一答案。
- 要區分答案是依公司公式算的，還是 Agent 自己湊的。後者不是不能用，但湊得越多，越可能是該轉成正式指標的訊號。

Schema 變更風險

- 好處：有 semantic layer 後，改 schema 只需對應改一處。
- 壞處：若沒同步，所有 Agent 呼叫會連鎖出錯，所以資料一變更就要立刻跟上。

Metadata 從文件變成資產

- 以前少寫沒更新，程式照跑；AI 時代沒定義好，後面整個會壞。

- 以後別人會問你的不只是「有沒有 API」，而是「有沒有足夠的 metadata 與 semantic layer，讓我的 Agent 能與你溝通」。

AI 時代治理框架要多出的能力：傳統 metadata 管理是寫完就放；AI 時代必須持續維護。（錄音在這裡結束，具體內容未錄到。）